A2 · Answer Key and Demo Script

CS425 · It Is Always DNS (Except When It Is Not)

INSTRUCTOR COPY

Graded pass/fail. Every transcript here is real output from a live testbed, so you can run these at the podium and get the same shapes. Set the two addresses first, from `a2-connectivity-triage.sh card`:

```
$ ALPHA=35.90.160.84        # yours will differ, every launch
$ BRAVO=18.236.210.3
```

HOW EACH STATION IS BROKEN

#	TARGET	MECHANISM	SIGNATURE
1	`ALPHA 8080`	Nothing wrong. Port allowed, server answers.	200, `connect=` one RTT
2	`ALPHA 8081`	Server *is* listening. The security group has no rule for 8081, so the SYN is dropped with no reply.	Hang, then timeout. Ping to ALPHA still works.
3	`ALPHA 8082`	Port allowed, nothing listening. Kernel answers RST.	Refused, effectively instant
4	`ghost.cs425.lab`	dnsmasq is authoritative for the zone and has no such record.	`status: NXDOMAIN`
5	`mirage.cs425.lab`	A record pointing at `10.42.13.37`, RFC 1918, not routable off their own LAN.	Resolves, then hangs like station 2
6	`BRAVO 8080`	Second host, second security group. TCP 8080 allowed, no ICMP rule at all.	Ping 100% loss, curl 200
7	`ALPHA 8084`	Accepts and never writes. **Held back for A3**; do not point the class at it today.	—

DEMO · STATIONS 1, 2 AND 3

Station 1 · the healthy signature

```
$ curl -sS -o /dev/null -m 8 -w 'connect=%{time_connect} total=%{time_total} code=%{http_code}\n' \
      http://$ALPHA:8080/
connect=0.022526 total=0.065459 code=200

$ ping -c 3 $ALPHA
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 21.009/22.100/22.936/0.807 ms
```

SAY: connect is 22.5 ms, the ping RTT is 21 ms. The handshake costs **ONE ROUND TRIP**: you send SYN, and `connect()` returns when the SYN-ACK lands. That is the number every other station gets compared against.

Station 2 · the silent drop

```
$ curl -sS -o /dev/null -m 6 -w 'connect=%{time_connect} total=%{time_total}\n' http://$ALPHA:8081/
curl: (28) Connection timed out after 6003 milliseconds
connect=0.000000 total=6.003387

$ nc -vz -G 6 $ALPHA 8081   # -G on macOS, -w on Linux
nc: connectx to 35.90.160.84 port 8081 (tcp) failed: Operation timed out
   ... 6.007 s total. With -w on a Mac this sits for about 75 s instead.

$ ping -c 2 $ALPHA   # the host is fine, prove it
2 packets transmitted, 2 packets received, 0.0% packet loss
```

SAY: `connect=0.000000`. The connection never happened, so there is no time to report. Nothing came back at all; curl retransmitted its SYN and gave up on its own clock. And ping still works, so the host is up. Whatever is wrong sits in front of *one port*.

Station 3 · the refusal, the same host seconds later

```
$ curl -sS -o /dev/null -m 6 http://$ALPHA:8082/
curl: (7) Failed to connect to 35.90.160.84 port 8082 after 25 ms: Couldn't connect to server

$ nc -vz -G 6 $ALPHA 8082
nc: connectx to 35.90.160.84 port 8082 (tcp) failed: Connection refused
   ... 0.029 s total.
```

SAY: 25 MILLISECONDS AGAINST 6000. Same host, same laptop, same network. That 25 ms is one round trip, so something answered immediately: a **TCP RST**. Note that curl on macOS says "Couldn't connect to server" while `nc` names the reset outright, so run `nc` if the room needs the word. This is the single most valuable moment of the period. **FLAG WARNING:** the connect timeout is `-G` on macOS and `-w` on Linux. Using `-w` on a Mac against station 2 hangs for about 75 seconds with no output, which at a podium looks like you broke something.

DEMO · STATIONS 4, 5 AND 6

Stations 4 and 5 · the DNS pair

```
$ dig ghost.cs425.lab @$ALPHA -p 5353
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 21746
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

$ dig +short mirage.cs425.lab @$ALPHA -p 5353
10.42.13.37

$ curl -sS -o /dev/null -m 6 http://10.42.13.37:8080/
curl: (28) Connection timed out after 6004 milliseconds

$ dig +short alpha.cs425.lab @$ALPHA -p 5353   # control: the zone works
35.90.160.84
```

SAY: Station 4 sent **ZERO PACKETS** to any target. There is no connectivity problem because there was never a destination. Station 5 resolved perfectly and still hangs, identically to station 2, because `10.0.0.0/8` is RFC 1918 and goes nowhere off their own LAN. **DIG IS THE ONLY PROBE THAT TELLS THOSE TWO APART.**

Station 6 · the probe that lies

```
$ ping -c 3 $BRAVO
3 packets transmitted, 0 packets received, 100.0% packet loss

$ curl -sS -o /dev/null -m 8 -w 'connect=%{time_connect} code=%{http_code}\n' http://$BRAVO:8080/
connect=0.025437 code=200
```

SAY: 100% loss, and the service is fine. ICMP is a **NETWORK LAYER** protocol riding directly in IP, and BRAVO's security group has no rule for it. Ask which host is misconfigured. Answer: neither. The difference lives *in front of* BRAVO, not on it.

ROUND 1 · THE HEALTHY SIGNATURE

Why are the ping RTT and curl's `connect=` in the same neighborhood?
The TCP handshake costs one round trip: the client sends SYN and `connect()` returns when the SYN-ACK arrives. Ping measures the same path with an ICMP echo. Accept anything saying "the handshake is one round trip over the same path".

PASS BAR: a real number in each probe column, not "it worked".

ROUND 2 · STATIONS 2 AND 3

Which is which, and what did the host send back in the case that failed fast?
Station 3 failed fast because the host sent a **TCP RST**. Something received the packet and answered "no". **Station 2 hung** because **nothing came back at all**: the SYN was dropped silently, the client retransmitted with exponential backoff and eventually gave up on its own timer.

A refusal is the more informative failure. A RST proves the address is routable, the host is up and its IP stack is running; it narrows the fault to one port on one machine. A timeout tells you almost nothing, because a drop can happen anywhere along the path.

Push on this if you hear it

- "**Station 3 means the server is down.**" The exact opposite. Chase this one; it is the most valuable correction available in the period.
- "**Station 2 is down, station 3 is up.**" Same host, and ping proves it is up in both cases.
- "**The firewall rejected it**" for station 2. Reject and drop are different behaviors. A filter that rejected would produce station 3's symptom, which is why choosing between them is a design decision.

PASS BAR: stations 2 and 3 are not described the same way, and one is identified as having produced no reply at all.

ROUND 2 · STATIONS 4 AND 5

Station 4: what was the status field, and how many packets reached the target?
NXDOMAIN, and **zero**. The failure happened before a single packet was addressed to anything, so there is no connectivity problem here at all. Layer: application (DNS). Fixed by whoever owns the zone.

Station 5: which one probe separates it from station 2, and what is special about the address?
dig, and it is the only one; curl, ping and nc all behave identically. The address is inside `10.0.0.0/8`, **RFC 1918 private space**, not routable across the public internet. Their packets go to their own default gateway and die, or reach an unrelated machine on their own LAN.

DNS succeeded and still caused the outage. "It resolved" is not "it resolved to something useful". Layer: application, with the symptom appearing at the network layer.

PASS BAR: station 4 is called a name problem rather than a connectivity problem, and station 5's row records the `10.x` address rather than only "it timed out".

ROUND 3 · THE PROBE THAT LIES

1. **Ping** reported it unreachable; **curl** proved it was not.
2. **ICMP**, at the **network layer**. It rides directly in IP, alongside TCP and UDP rather than on top of them. Accept "layer 3" or "same layer as IP".
3. The **security group**: ALPHA's allows ICMP, BRAVO's has no ICMP rule. It lives **in front of** the host. Nothing on BRAVO itself differs, and its kernel would answer the echo if the packet ever arrived. "BRAVO has ICMP turned off in its OS" has the right shape and the wrong location; correct it out loud, because it is the difference between debugging the host and debugging the path.
4. Something shaped like: "*Not answering ping only tells us ICMP is filtered. Plenty of production hosts drop it on purpose. Let me try the actual port before we page anyone.*"

PASS BAR: ping is named as the probe that lied, and ICMP is placed at the network layer.

IF THEY FINISH EARLY · THE DNS INTERCEPTION CHECK

Drop `-p 5353` and compare.
On a network that hijacks outbound port 53, the plain query is answered by the interceptor rather than by ALPHA, and an interceptor that has never heard of `cs425.lab` answers **NXDOMAIN for everything**. That makes station 4 look right and station 5 look broken, which is the most confusing possible way for this to fail.

This happened on the machine this activity was built on, so it is worth expecting. `verify` detects it and warns you. If you see that warning, tell the room: they are sitting on a live middlebox lying to them, and it is the same class of bug as station 5 one layer up.

Debrief, in this order

1. **Station 3 proves the host is alive.** Refusal is information; timeout is the absence of it. Sets up the TCP state machine in chapter 3.
2. **Station 5 is a DNS bug wearing a network bug's clothes.** Sets up RFC 1918 and NAT in chapter 4.
3. **Station 6 means ping tests ICMP, not reachability.** Sets up firewalls in chapter 8.

Grade these and hand them back at the start of A3. A3 is built on the station table, and a group that arrives without data spends the period re-probing instead of thinking. Pass/fail on a flip-through is quick enough to do this between the two class meetings.

Before each section

Run `./scripts/cs425/a2-connectivity-triage.sh verify`. It asserts every signature above from your machine in about twenty seconds, and it is how you find out a security group drifted before thirty students do. A clean run is **9 passed, 0 failed**. Leave the testbed up for A3, or relaunch and put the new addresses on the board.